Addis Ababa University
College of Natural and Computational Science
Department of Physics

Computational Physics Application

(Phys 4624)

Simulation of the Boltzmann Distribution using the Metropolis Algorithm

Name: Getabalew Manegerew
ID Number: UGR/6981/15
Instructor: Dr. Lemi D.

May 26, 2026

1 Introduction

The objective of this project is to simulate the behavior of a single classical particle in one dimension, in thermal equilibrium with a heat bath at temperature T. I aim to show that the Metropolis algorithm correctly generates the Boltzmann distribution:

$$P(E) = \frac{e^{-\beta E}}{Z} \tag{1}$$

where $\beta = \frac{1}{k_B T}$, E is the energy of the system, and Z is the partition function. For this simulation, I consider the given assumption of that, particle mass $m = 1$ with energy $E = \frac{1}{2}v^2$, and $k_B = 1$.

2 Methodology

2.1 The Metropolis Algorithm

The simulation follows the Metropolis-Hastings logic to sample the equilibrium distribution:

1. **Initialization:** Start with an initial velocity v_0.
2. **Proposal:** Propose a new velocity $v_{new} = v_{old} + \delta v \cdot r$, where r is a random number in $[-1, 1]$. δv is choosen in such a way to have an acceptance of somewhere in the $60 - 70\%$.
3. **Cutting Outliers:** If $v_{new} \leq v_{max}$, the next step followed, otherwise new trail velocity proposed again. i.e. $v_{max} = 10\sqrt{\frac{1}{\beta}}$
4. **Probability Ratio:** Calculate the ratio of the probability of the system finding in the new and old state according to boltzmann distribution ($e^{-\beta \Delta E}$ for $\Delta E = E(v_{new}) - E(v_{old}) = \frac{1}{2}(v_{new}^2 - v_{old}^2)$.
5. **Acceptance Criteria:**
   - If ratio > 1, accept the new velocity.
   - If ratio < 1 or $\Delta E \leq 0$, accept with probability ratio $= e^{-\beta \Delta E}$. This is done, by picking a random number and comparing it with the ratio.
6. **Iteration:** Repeat for a total of N Monte Carlo (MC) steps, i.e. $N = 10000$ as required.

2.2 Implementation Parameters

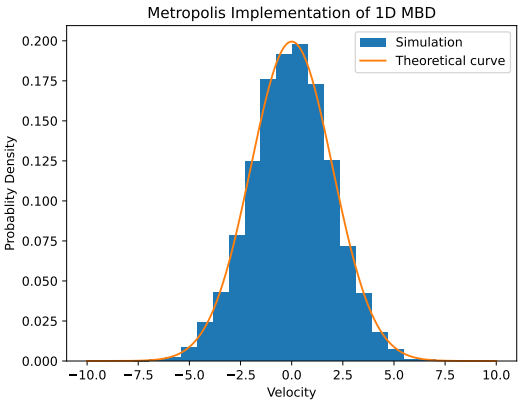
The simulation is performed using the following parameters as specified:

- **Temperature (T):** 4.0
- **Initial Velocity (v_0):** 0.0 (and 2.0 for testing dependency)
- **Velocity Change Step (δv):** 4.0
- **MC Steps:** 10,000
- **Histogram Bins (N_{bin}):** 20, 30, 50

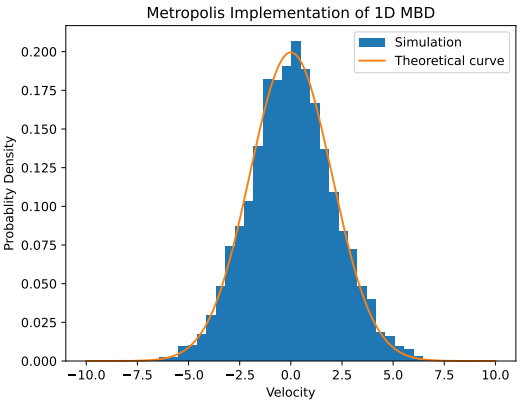
3 Results

3.1 Velocity Probability Density $P(v)$

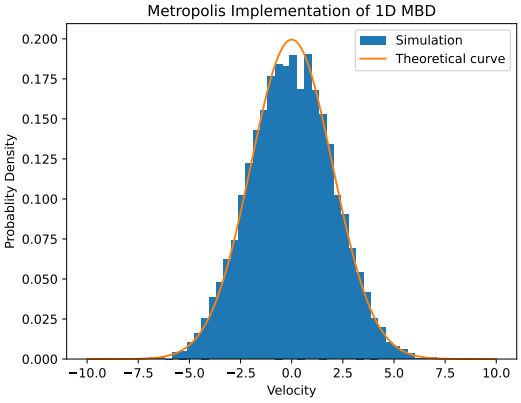
After running the MC cycles, the velocities were collected into a histogram to represent $P(v)dv$.



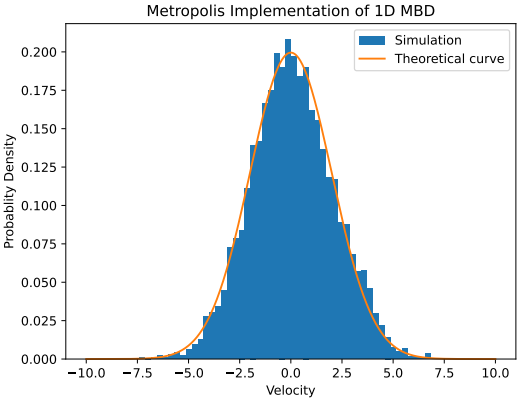
(a) 20 bins



(b) 30 bins



(c) 40 bins



(d) 50 bins

Figure 1: Normalized histograms of sampled velocities vs. theoretical Boltzmann distribution for various bin sizes.

3.2 Statistical Moments

The calculated mean energy and mean velocity are compared to the analytical expectations:

Metric	Simulated Value	Analytical Value
Mean Velocity $\langle v \rangle$	-0.014587	0
Mean Energy $\langle E \rangle$	1.990761	$\frac{1}{2}T = 2.0$

Table 1: Comparison of simulation statistics with exact physical values.

4 Analysis and Discussion

4.1 Functional Form and Energy

From the graphical outputs of Figure 1, the probability distribution seemed to mimic Maxwell-Boltzmann's velocity distribution. Indeed theoretically, we expect $P(v)$ to be this distribution.

From MBD we know $P(v) \propto e^{-\beta E}$, hence in semilog plots of E versus P a linear curve is expected. The normalized distribution is given as follows,

$$P(E) = \sqrt{\frac{m\beta}{2\pi}} e^{-\beta E} \tag{2}$$

The result of my simulation demonstrated the expected fact that, the slop of the semilog plot has a slop of $-\frac{1}{\beta}$. Which is $\frac{-1}{4}$ in the given case. This is shown in Figure 2.

4.2 Logarithmic Verification

To verify the distribution form, we plot $\ln(P(v))$ against the energy E.

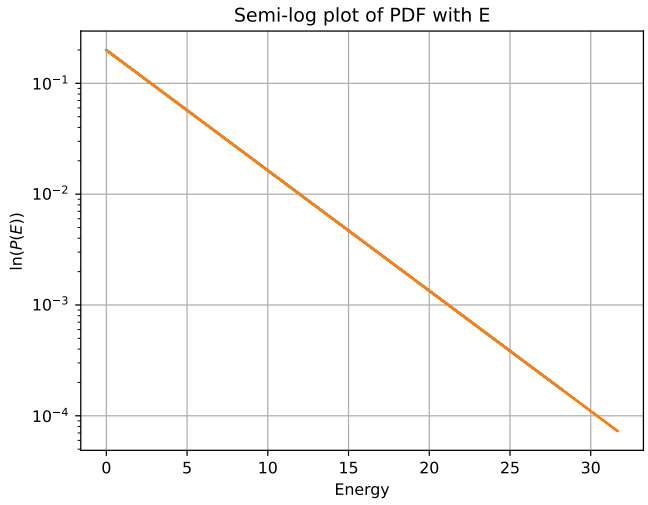


Figure 2: Linearized Boltzmann distribution. The slope should approximate $-1/T = -0.25$.

4.3 Initial Condition Sensitivity

The simulation was repeated with $v_0 = 2$ to ensure results are independent of initial conditions. The results seemed to be independent of the initial value of v, as we may expect from metropolis algorithm. there is no a marked shift both on the statistical value of mean energy and mean velocity of the system as well as a qualitative features of the curves remain the same.

Hence, this verifies the fact that, our method is independent of the iniitial configuration of the system.

5 Conclusion

In this project I simulated a 1D particle using metropolis algorithim. The weight function was $e^{-\beta E}$, which is boltezmann factor of MBD. The results also seemed to achieve this distribution.

More importantly, statistically expected values of mean energy and mean velocity are in great agreement with theoretical expectation. In addition to that, graphical outputs approached MBD results with a remarkable closeness. The result was also marked independence of the initial state of the system.

A Source Code

```
# -*- coding: utf-8 -*-
"""
Created on Tue May  5 19:05:54 2026

@author: getsh

General Comment:
This script implements the Metropolis-Hastings algorithm to simulate a 1D Boltzmann
velocity distribution. It samples the probability density function P(v) proportional
to exp(-beta * v^2 / 2) and compares the resulting numerical histogram against the
analytical theoretical curve.
"""

import numpy as np
import matplotlib.pyplot as plt

# Simulation parameters
v_delta = 4 # Maximum step size for velocity proposal
v0 = 2 # Initial velocity of the particle
beta = 1/4 # Inverse temperature (1/kT)
v_max = 5*np.sqrt(1/beta) # Maximum velocity bound for the simulation range

def boltz_fun(x):
    # Returns the unnormalized Boltzmann factor for a given velocity
    return np.exp(-beta*x**2/2)

N=10000 # Total number of Monte Carlo steps
vel = np.zeros(N) # Array to store velocity history
accept = 0 # Counter for accepted moves
n_used = 0 # Counter for steps within the valid velocity range

# Metropolis Algorithm Loop
for i in range(N):
    # Propose a new velocity using a uniform distribution displacement
    v_trial = v0 + v_delta*(2*np.random.uniform() - 1)
    # Calculate the acceptance ratio (P_new / P_old)
    ratio = boltz_fun(v_trial)/boltz_fun(v0)
    r = np.random.uniform()

    # Check if trial move is within velocity bounds
    if (v_trial**2 <= v_max**2):
        n_used +=1
```

```
        if (ratio >= r or (v_trial**2 - v_0**2) <=0): # accept the step
            vel[i] = v_trial
            v0 = v_trial
            accept +=1
        else: # rejecting the step and stay there
            vel[i] = v0

# Data Analysis
E = 1/2*vel**2 # the energy of the distribution at each configuration as an
    array
E_mean = np.mean(E) # the mean value of the energy
v_mean = np.mean(vel) # the mean value of the velocity distribution

# Theoretical Curve Preparation
v = np.linspace(-v_max, v_max, n_used) # this creates values of velocity in [-
    v_max, v_max] range
pdf = boltz_fun(v)*np.sqrt(beta/(2*3.14)) # this mimic the actual theoretical
    function

# Output results
print("Acceptance rate:", accept/n_used, E_mean, v_mean)

# Visualization
plt.hist(vel, bins=50, density = "True", label="Simulation") # this is to mimic
    theoretical curve of MBD
plt.plot(v, pdf, label="Theoretical curve")
plt.xlabel("Velocity")
plt.ylabel("Probablity Density")
plt.title("Metropolis Implementation of 1D MBD")
plt.legend()

"""
Analysis Section: Linearization of the Boltzmann distribution
pdf = boltz_fun(vel)*np.sqrt(beta/(2*np.pi))
plt.semilogy(E, pdf)
plt.xlabel("Energy")
plt.ylabel(r"$\ln(P(E))$")
plt.title("Semi-log plot of PDF with E")
plt.grid()
"""
```

Listing 1: Python implementation of the Metropolis Algorithm for Boltzmann Sampling